

[LavaGo]
(SAD) Software Architecture Document
Versión 1.7

Identificación de Documento

Identificación	SAD (Software Architecture Document)
Proyecto	Sistema de Gestión de Inventario de Ropa Hospitalaria
Versión	1.7

Documento mantenido por	Equipo de desarrollo
Fecha de última revisión	03/11/2024
Fecha de próxima revisión	22/10/2024

Documento aprobado por	Yamilet Maureira (Líder de proyecto)
Fecha de última aprobación	03/11/2024

Historia de Revisiones

Fecha	Versión	Descripción	Autor
01/10/2024	1.0	Se completó la identificación del documento y el punto 1.1 de la introducción.	Equipo de desarrollo
12/20/2024	1.1	Finalización de las secciones 1.2 a 1.7 de la introducción.	Equipo de desarrollo
18/10/2024	1.2	Revisión y ajustes realizados en la sección 1.7.	Equipo de desarrollo
20/10/2024	1.3	Finalización de los puntos de la sección 2 y comienzo de la sección 3.	Equipo de desarrollo
21/10/2024	1.4	Se completaron los puntos 3.2 y 3.3 de la sección 3.	Equipo de desarrollo
22/10/2024	1.5	Se han incorporado imágenes del “Diagrama de Clases”, “Diagrama de Comunicación” y “Vista Física” en la sección 3.	Equipo de desarrollo
25/10/2024	1.6	Finalización de las secciones 4 y 5.	Equipo de desarrollo
01/11/2024	1.7	Se han incorporado imágenes del “Diagrama de secuencia”, “Vista de Procesos” y “Vista de Despliegue” en la sección 3.	Equipo de desarrollo

Tabla de Contenidos

1	INTRODUCCIÓN	3
1.1	CONTEXTO DEL PROBLEMA	3
1.2	PROPÓSITO	3
1.3	ÁMBITO	3
1.4	DEFINICIONES, ACRÓNIMOS Y ABREVIACIONES	4
1.5	REFERENCIAS	5
1.6	RESUMEN EJECUTIVO	6
1.7	REPRESENTACIÓN	6
2	METAS Y RESTRICCIONES DE LA ARQUITECTURA	9
2.1	METAS DE LA ARQUITECTURA	9
2.2	RESTRICCIONES DE LA ARQUITECTURA	10
2.3	OTROS ANTECEDENTES Y CONSIDERACIONES	10
3	VISTA DE CASOS DE USO Y ESCENARIOS DE CALIDAD	11
3.1	MODELO DE CASOS DE USO	11
3.2	ESPECIFICACIÓN DE CASOS DE USO RELEVANTES	11
3.3	ESPECIFICACIÓN DE LOS ESCENARIOS DE CALIDAD RELEVANTES	13
	VISTA LÓGICA	19
	VISTA DE PROCESOS	30
	VISTA DE DESPLIEGUE	31
	VISTA FÍSICA	33
	VISTA ESCENARIOS 4+1	34
4	DECISIONES DE DISEÑO Y SELECCIÓN DE ALTERNATIVAS	35
5	ANÁLISIS DE REUTILIZACIÓN	36

● **Introducción**

●.1 **Contexto del Problema**

En el entorno hospitalario, la gestión eficiente del inventario de ropa es crucial para garantizar la calidad del servicio y la atención al paciente. Sin embargo, el hospital enfrenta dificultades significativas para llevar un control adecuado de la ropa hospitalaria, lo que puede resultar en escasez o exceso de prendas, afectando la operatividad de los servicios clínicos.

El desafío principal radica en la falta de un sistema integral que permita registrar y monitorear de manera efectiva los ingresos y egresos de ropa limpia y sucia, tanto en los servicios clínicos como en las lavanderías externas. Por ello, se requiere el desarrollo de una aplicación web que facilite el registro de los movimientos de ropa, generando reportes precisos y en tiempo real sobre el estado del inventario. Además, es fundamental asegurar que el sistema sea intuitivo y accesible desde diversos dispositivos, garantizando su disponibilidad continua y su capacidad de respuesta durante períodos de alta carga.

●.2 **Propósito**

El propósito de este documento es detallar el diseño y desarrollo de una aplicación web destinada a mejorar la gestión del inventario de ropa hospitalaria. Este sistema tiene como objetivo principal abordar las deficiencias actuales en el control de entradas y salidas de prendas en el hospital San José de Melipilla, asegurando un registro preciso y eficiente de la ropa limpia y sucia en los servicios clínicos y en las lavanderías externas.

La importancia de este proyecto radica en su potencial para transformar la gestión de la ropa hospitalaria en un proceso más eficiente y efectivo, contribuyendo significativamente a la calidad del servicio y a la atención al paciente. Al implementar esta solución, se espera no solo mejorar la operatividad diaria, sino también sentar las bases para un sistema escalable que pueda adaptarse a las necesidades futuras del hospital.

●.3 **Ámbito**

El proyecto abordará funciones clave como; el registro eficiente de ingresos y egresos de ropa limpia y sucia en los servicios clínicos y en la lavandería externa, la generación de reportes detallados sobre el estado del inventario, notificaciones sobre escasez de ropa, y la garantía de seguridad de los datos, asegurando que solo usuarios autorizados tengan acceso a la información.

La aplicación será utilizada por personal de enfermería, administradores de inventario y personal de lavandería, y se implementará en los servicios clínicos del hospital y en la lavandería externa asociada, garantizando su accesibilidad en diferentes entornos operativos.

●.4 Definiciones, acrónimos y abreviaciones

DEFINICIONES	DESCRIPCION
Aplicación Web	Sistema accesible a través de un navegador que permite a los usuarios interactuar con el software.
Modularidad	Diseño del sistema en componentes independientes que pueden ser desarrollados y mantenidos por separado.
Uptime	Tiempo durante el cual un sistema está operativo y disponible para su uso, expresado como un porcentaje.
Offline	Capacidad de un sistema o aplicación para funcionar sin una conexión activa a internet.
Atributos	Características o propiedades específicas de cada entidad dentro del sistema, que ayudan a describir y definir su estado en un momento dado.
Métodos	Funciones o procedimientos asociados a cada clase que definen su comportamiento.
Controlador	Componente que gestiona la interacción entre el usuario y la lógica de negocio, procesando las entradas del usuario y coordinando las respuestas adecuadas.
Objetos	Entidades en el sistema que encapsulan datos y comportamientos. Cada objeto es una instancia de una clase, con atributos que describen su estado y métodos que definen sus acciones.
Frontend	Parte del sistema que interactúa directamente con el usuario. Incluye la interfaz gráfica y la lógica de presentación.
Backend	Parte del sistema que se ejecuta en el servidor y se encarga de la lógica de negocio, la gestión de bases de datos y la comunicación con el frontend.
Performance	Eficiencia de un sistema en ejecutar tareas y responder solicitudes, abarcando el tiempo de respuesta, el uso de recursos, la escalabilidad y la capacidad de gestionar múltiples procesos simultáneamente.
ACRÓNIMOS	DESCRIPCION
SAD	Software Architecture Document: Documento que detalla el diseño y desarrollo del sistema.
API	Application Programming Interface: Interfaz que permite la interacción entre diferentes sistemas.
REST	Representational State Transfer: Estilo arquitectónico para diseñar redes de aplicaciones que utilizan HTTP.

BLoC	Business Logic Component: Patrón de diseño utilizado en Flutter para separar la lógica de negocio de la interfaz de usuario.
UML	Unified Modeling Language: Lenguaje de modelado unificado utilizado para especificar, visualizar y documentar artefactos de software.
GDPR	General Data Protection Regulation: Reglamento de protección de datos de la unión europea que establece pautas para la recopilación y el procesamiento de información personal.
HTTP	Hypertext Transfer Protocol: Protocolo de comunicación utilizado para la transferencia de datos en la web. Permite que los navegadores y servidores se comuniquen entre sí.
HTTPS	Hypertext Transfer Protocol Secure: Utiliza cifrado para proteger la transferencia de datos entre el navegador y el servidor. Esto significa que la información intercambiada, como credenciales de inicio de sesión, se mantengan seguras y privadas.
JSON	JavaScript Object Notation: Formato ligero y legible para el intercambio de datos, que utiliza una estructura de pares de clave-valor y arreglos.
ABREVIACIONES	DESCRIPCION
E/S	Entradas/Salidas: Control de ingresos y egresos de ropa limpia y sucia.
BD	Base de Datos: Conjunto estructurado de datos que permite almacenar, gestionar y recuperar información de manera eficiente.

●.5 Referencias

A continuación se listan las referencias a otros documentos:

1. Documentos de proyectos relacionados:

- "Metas y Restricciones de la arquitectura." *studylib.es*.
<https://studylib.es/doc/383874/3.-metas-y-restricciones-de-la-arquitectura> (2010).

2. Documentos internos:

- "Casos de uso" (2024).
- "Plantilla escenarios de calidad" (2024).

3. Sitios Web:

- [Documentar la arquitectura del software: la guía completa | AppMaster](#)
- [Cómo cumplir con las normativas GDPR, HIPAA y PCI-DSS en la nube \(geko.cloud\)](#)

- <https://boardmix.com/es/articles/use-case-examples/>
- <https://diagramasuml.com/diagrama-de-clases/>
- <https://www.edrawsoft.com/es/article/communication-diagram-uml.html>
- <https://miro.com/es/diagrama/que-es-diagrama-despliegue-uml/>
- [FLUTTER: PROVIDER PARA PRINCIPIANTES -GUÍA DE INICIO | by Saul Ram | Medium](#)
- [Hot reload | Flutter](#)
- [CON_NAC_06.pdf](#)
- [JSON | Flamear](#)
- [▷ Diagrama de componentes. Teoría y ejemplos](#)
- [▷ Diagrama de paquetes. Teoría y ejemplos](#)

●.6 Resumen ejecutivo

El documento **(SAD)** detalla el diseño y desarrollo de un sistema de gestión de inventario de ropa hospitalaria, con el objetivo de optimizar el control de la ropa en el entorno clínico y mejorar la atención al paciente. En la Introducción, se establece el contexto del problema, subrayando la necesidad de un sistema integral para registrar eficientemente las **(E/S)** de ropa limpia y sucia.

Las metas y restricciones de la arquitectura se centran en aspectos como el desempeño, la seguridad y la escalabilidad, mientras que las vistas de casos de uso y escenarios de calidad describen funcionalidades críticas del sistema, junto con los requisitos de calidad que deben cumplirse.

Las decisiones de diseño y selección de alternativas, enfatizan la modularidad y la capacidad de recuperación ante fallos, para asegurar la operatividad del sistema bajo diversas condiciones. Y por último, el análisis de reutilización, destaca cómo los componentes existentes pueden ser aprovechados para optimizar el desarrollo del sistema.

●.7 Representación

La arquitectura del sistema de gestión de inventario de ropa hospitalaria está representada siguiendo el enfoque del framework 4+1 y las recomendaciones del proceso unificado. Las vistas incluidas en esta versión del documento son:

1. **Vista de Casos de Uso y Escenarios de Calidad:** Describe la interacción entre los actores (**usuarios o sistemas externos**) y los casos de uso del sistema. Se representa mediante diagramas UML que ilustran las funcionalidades del sistema y los escenarios de calidad asociados, como rendimiento, seguridad y usabilidad. Los escenarios de calidad se detallan

para garantizar que los requisitos no funcionales se cumplan adecuadamente.

2. **Vista de Metas y Restricciones:** Define las metas arquitectónicas que el sistema debe alcanzar, tales como desempeño, escalabilidad y seguridad. También, se identifican las restricciones que pueden afectar el diseño, como normativas de protección de datos (**GDPR**) y requisitos tecnológicos (**compatibilidad multiplataforma**). Esta información es fundamental para guiar las decisiones de diseño y asegurar que el sistema cumpla con los estándares establecidos.
3. **Vista Lógica:** Representa la estructura del sistema a un nivel abstracto, mostrando los componentes principales, sus relaciones y cómo interactúan entre sí. Esta vista es clave para entender la arquitectura modular del sistema y cómo se gestionan los datos.
 - Diagrama de clases: Proporciona una representación estática de las entidades del sistema, sus atributos y las relaciones que las vinculan. Este diagrama es crucial para el modelo de dominio del sistema, donde se definen clases clave como **“Ropa”**, **“Servicio clínico”**, **“Lavandería”** y **“Usuario”**. A través de la visualización de asociaciones, generalizaciones y composiciones, se establece cómo se estructuran los datos y cómo interactúan los objetos en el contexto del sistema. Además, permite identificar los métodos asociados a cada clase, facilitando así la implementación y el mantenimiento del sistema.
 - Diagrama de secuencia: Representa el orden de las interacciones entre los objetos en un escenario específico, evidenciando la transmisión de mensajes a lo largo del tiempo. Por ejemplo, modelar el proceso de **registro de ingresos de ropa**, detallando la secuencia de invocaciones entre componentes como el controlador, el servicio de negocio y la BD. Es esencial para visualizar el flujo de control dentro del sistema, permitiendo identificar claramente el tiempo requerido para cada interacción y la dinámica de la comunicación entre los distintos módulos. Al hacerlo, facilita la comprensión de la lógica de procesamiento y la optimización de los tiempos de respuesta en el sistema.
 - Diagrama de comunicación: También conocido como **“Diagrama de colaboración”**, se centra en las interacciones entre los objetos y la forma en que se comunican para ejecutar una tarea específica. A diferencia del **“Diagrama de secuencia”**, que enfatiza el orden temporal de los mensajes, este diagrama pone de relieve la estructura de los vínculos entre los objetos, ilustrando cómo están interconectados y cómo intercambian información para alcanzar un objetivo común. Por ello, puede modelar procesos críticos, como el registro de ingresos y egresos de ropa, permitiendo no solo visualizar las relaciones entre componentes, sino que también facilitando la identificación de dependencias y puntos de interacción.
4. **Vista de Procesos:** Describe cómo el sistema ejecuta funcionalidades a través de procesos,

que son unidades de trabajo que agrupan tareas. Se enfoca en **performance**, **disponibilidad** y **tolerancia a fallas**, permitiendo la ejecución simultánea y la distribución de carga en hardware. Facilita el control de procesos (iniciar, detener y recuperar) y su replicación para mejorar la disponibilidad, además de separar operaciones en línea y fuera de línea para pruebas, asegurando un funcionamiento eficiente y la coexistencia de diferentes versiones de software.

- Diagrama de actividad: Es un “**Diagrama de comportamiento**” que representa el flujo de control y objetos en el sistema, centrándose en la secuencia y condiciones de flujo. Ilustra actividades clave, como el registro de ingresos y egresos de ropa, y destaca las interacciones del personal, así como decisiones críticas sobre la validez de los datos ingresados. Además, considera eventos externos que pueden activar acciones, lo que refleja la flexibilidad del sistema ante situaciones cambiantes. Este diagrama es fundamental para comprender la eficiencia del sistema, identificar áreas de mejora y garantizar el cumplimiento de los requisitos de rendimiento y disponibilidad.
5. **Vista de Despliegue**: Muestra la distribución física de los componentes del sistema. Identifica los nodos clave, como el frontend, backend y BD, cada uno en servidores específicos. Se detallan las interconexiones entre nodos, los protocolos de comunicación (**HTTP/HTTPS**) y de escalabilidad. También, se abordan medidas de seguridad, así como herramientas de monitoreo para asegurar la disponibilidad y rendimiento del sistema.
- Diagrama de componentes y paquetes: Ofrece una representación visual de la arquitectura modular del sistema, mostrando los **componentes** clave, como los **módulos** de registro de ingresos y egresos, así como la generación de reportes, organizados en **paquetes** que reflejan su funcionalidad y cohesión. Cada componente está diseñado para encapsular funciones específicas, facilitando el mantenimiento y la escalabilidad. Por ejemplo, el componente de la **API** gestiona la comunicación entre el frontend y el backend, mientras que el módulo de la base de datos se encarga del almacenamiento y recuperación de información crítica.

El diagrama también muestra las relaciones entre los componentes, destacando las **dependencias** y las **interfaces** necesarias para la integración. Esto es fundamental para garantizar un manejo eficiente y seguro de los flujos de información, como los registros de ropa limpia y sucia. Además, proporciona a los desarrolladores una comprensión clara de la arquitectura general, permitiendo identificar áreas para posibles mejoras o modificaciones futuras.

6. **Vista Física**: Se centra en los recursos físicos necesarios para la implementación del sistema detallando la infraestructura necesaria para su funcionamiento óptimo. Incluye

especificaciones sobre servidores, almacenamiento y dispositivos de red. Esta vista es esencial para planificar la infraestructura y garantizar que se cumplan los requisitos de capacidad y rendimiento.

- Diagrama de despliegue: Se comporta como una representación estática de la arquitectura física del sistema, mostrando cómo se distribuyen los **componentes** en el entorno de implementación. Este diagrama ilustra las relaciones entre los **nodos**, como servidores y dispositivos de red, y los componentes de software, permitiendo visualizar cómo interactúan entre sí. Asegura que **módulos**, como la gestión de ingresos y egresos, estén desplegados de manera eficiente para cumplir con los estándares de rendimiento y disponibilidad requeridos en un entorno operativo real.

• Metas y Restricciones de la Arquitectura

•.1 Metas de la arquitectura

De acuerdo a las reuniones y al análisis de los requerimientos, se listan los principales conductores iniciales de la arquitectura los cuales corresponden a las metas arquitectónicas iniciales:

1. Desempeño:

El sistema debe garantizar tiempos de respuesta inferiores a **2 segundos** en operaciones críticas, como el registro de ingresos y egresos de ropa. Esto incluye la capacidad de manejar un alto volumen de transacciones simultáneas sin degradar la experiencia del usuario.

2. Seguridad:

Se implementarán mecanismos de seguridad robustos, que incluirán autenticación y autorización de usuarios, así como encriptación de datos tanto en reposo como en tránsito. El acceso a la información sensible estará restringido a usuarios autorizados mediante roles y permisos definidos.

3. Operatividad:

La aplicación deberá ser accesible en todo momento (**99.9% de uptime**) y compatible con múltiples plataformas (**Windows, Linux, MacOS, Android, iOS**). Se garantizará que la interfaz sea intuitiva y fácil de usar, optimizando la experiencia del usuario final.

4. Integración:

Se implementarán **APIs RESTful** para facilitar la integración con otros sistemas y servicios externos como plataformas de envío de notificaciones (**SendGrid**).

●.2 Restricciones de la Arquitectura

Existen restricciones que han sido levantadas con los stakeholders, las cuales se presentan a continuación:

1. **Tiempo de construcción:** El desarrollo del sistema de gestión de inventario de ropa hospitalaria debe ser completado en un período máximo de **4 semanas**. En donde, el cronograma abarca todas las fases del ciclo de vida del desarrollo de software, incluyendo el análisis de requisitos, diseño de la arquitectura, codificación, pruebas y despliegue.
2. **Normativas de Seguridad:** El sistema debe cumplir con las normativas de protección de datos del sector salud, lo que implica implementar medidas estrictas de autenticación, autorización y encriptación. Los datos sensibles sólo deben ser accesibles por usuarios autorizados, lo que añade una capa adicional de complejidad en el diseño.
3. **Compatibilidad Multiplataforma:** La aplicación debe ser accesible desde múltiples plataformas, incluyendo diferentes sistemas operativos y navegadores. Esto limita ciertas elecciones tecnológicas y requiere un diseño responsivo que funcione bien en diversas pantallas y dispositivos.
4. **Conectividad de Red:** La aplicación debe ser funcional en entornos con conectividad variable. Esto puede limitar el uso de tecnologías que requieren una conexión a Internet constante y puede requerir la implementación de soluciones de almacenamiento en **caché** o funcionalidad **offline**.
5. **Mantenimiento y Soporte:** Cualquier solución adoptada debe permitir actualizaciones regulares y fácil resolución de problemas en el sistema.

●.3 Otros antecedentes y consideraciones

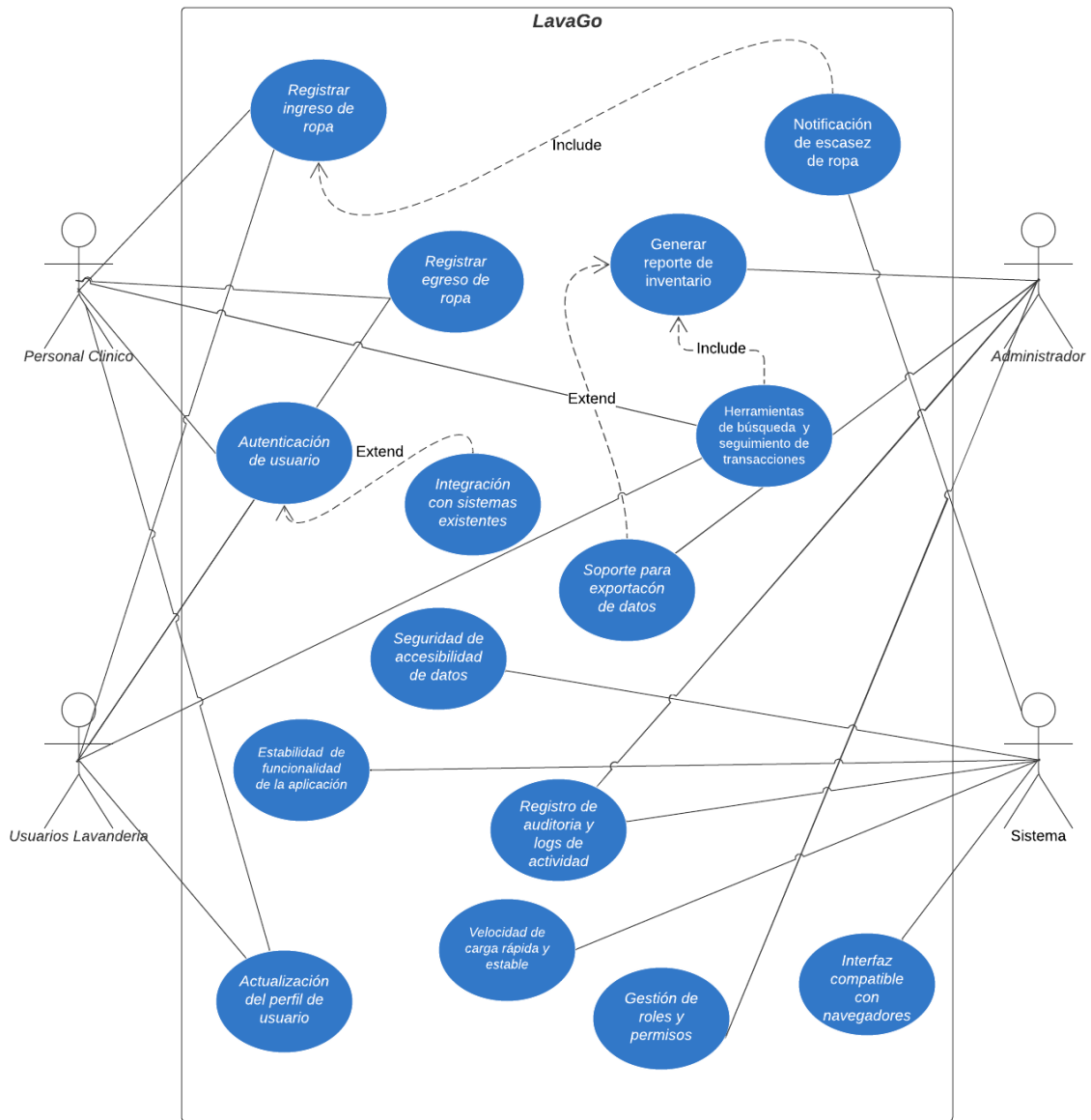
1. **Framework de Desarrollo:** Se optará por **Flutter** como framework de desarrollo multiplataforma, permitiendo la creación de una interfaz de usuario nativa para diversas plataformas (**iOS, Android, Web**) utilizando **Dart** como lenguaje de programación. Esto facilitará un desarrollo más ágil y una experiencia de usuario consistente.
2. **Modularidad y Pruebas:** La arquitectura seguirá principios de diseño modular, utilizando patrones como **BLoC** para gestionar el estado de la aplicación. Esto no solo promueve la reutilización de componentes, sino que también facilita las pruebas unitarias y de integración.
3. **Widgets Personalizados:** Se desarrollarán widgets personalizados en Flutter para la gestión

del inventario, asegurando que los componentes sean reutilizables en diferentes partes de la aplicación. Esto reducirá el tiempo de desarrollo y mantendrá una coherencia visual en la interfaz.

- **Vista de Casos de Uso y Escenarios de Calidad**

Esta sección describe en detalle el conjunto de escenarios funcionales y no funcionales que obtuvieron la mayor prioridad en el análisis. Para esto se presenta y describe el diagrama de casos de uso y los casos de uso prioritarios, así como los escenarios en que uno o más atributos de calidad se ven involucrados de manera significativa.

•.1 Modelo de Casos de Uso



•.2 Especificación de Casos de Uso Relevantes

La identificación y especificación de los casos de uso son pasos fundamentales en el diseño del sistema de gestión de inventario de ropa hospitalaria ya que, representan las interacciones entre los usuarios y el sistema, permitiendo visualizar cómo se llevarán a cabo las operaciones y cómo se cumplirán los requisitos funcionales.

En este contexto, se han seleccionado los casos de uso más relevantes en función de varios criterios, incluyendo la complejidad de la implementación, el riesgo asociado y su impacto en los atributos de calidad del sistema. Aunque, estos no solo reflejan las funcionalidades críticas que el sistema debe ofrecer, sino que también sirven como base para el desarrollo, las pruebas y la validación del software.

Código	Nombre	Actores	Prioridad
CU-001	Registrar ingreso de ropa	1. Personal Clínico 2. Sistema de notificaciones	Alta
CU-002	Autenticación de usuario	1. Sistema 2. Usuario registrado	Alta
CU-003	Registrar ingreso y egreso de ropa en lavandería externa	1. Personal de lavandería	Alta
CU-004	Generar reporte de inventario	1. Administrador 2. Sistema	Alta
CU-005	Notificaciones de escasez de ropa	1. Sistema 2. Usuarios autorizados	Alta
CU-006	Herramienta de búsqueda y seguimiento de transacciones	1. Usuarios autorizados	Media
CU-007	Integración con sistemas existentes	1. Sistema	Alta
CU-008	Soporte para exportación de datos	1. Administrador 2. Sistema	Media
CU-009	Seguridad de accesibilidad de datos	1. Sistema 2. Usuario registrado	Muy Alta
CU-010	Registro de auditoría y logs de actividad	1. Sistema 2. Administrador	Muy Alta
CU-011	Estabilidad de funcionalidad de la aplicación	1. Sistema	Muy Alta
CU-012	Interfaz compatible con navegadores	1. Sistema 2. Usuarios autorizados	Media

CU-013	Velocidad de carga rápida y estable	1. Usuarios del sistema	Alta
CU-014	Actualización del perfil de usuario	1. Usuario registrado	Media
CU-015	Gestión de roles y permisos	1. Administrador 2. Sistema	Alta

●.3 Especificación de los Escenarios de Calidad Relevantes

En el desarrollo de sistemas, la calidad abarca no solo la funcionalidad, sino también una serie de atributos no funcionales críticos para el rendimiento y la fiabilidad del sistema. Los escenarios de calidad son herramientas fundamentales que permiten especificar y medir estas características, estableciendo métricas claras sobre el comportamiento del sistema en diversas condiciones operativas.

En el contexto de la aplicación web destinada a la gestión del inventario de ropa hospitalaria, es imperativo asegurar que el sistema no solo efectúe registros precisos de ingresos y egresos, sino que también mantenga un rendimiento óptimo bajo condiciones de carga variable, garantizando la disponibilidad y la integridad de los datos. Es por ello que, a continuación, se presentarán los escenarios de calidad relevantes que guiarán el desarrollo y la evaluación del sistema:

Escenario de Calidad N°1	
ID	QS1
Nombre	Seguridad: Registro de auditoría y logs de actividad.
Sinopsis	Implementar registros de auditoría y logs de actividad para realizar un seguimiento detallado de las acciones en el sistema, proporcionando información en tiempo real sobre qué usuario ejecutó cada acción.
Entorno	Operación normal del sistema, donde los usuarios registran acciones relacionadas con la gestión de ropa limpia y sucia.
Cambio en el entorno	Un aumento en la carga de usuarios accediendo simultáneamente al sistema, lo que podría incrementar el volumen de acciones registradas y la cantidad de logs de auditoría generados,
Comportamiento esperado	El sistema debe registrar en tiempo real todas las acciones realizadas por los usuarios, garantizando que los logs de auditoría sean

	accesibles para los administradores autorizados.
Medida	1. Tiempo de registro de actividad: El sistema debe registrar una acción en un tiempo ≤ 1 segundo bajo condiciones normales. En condiciones de carga creciente (10, 50 y 100 usuarios simultáneos), el tiempo de registro no debe exceder 2 segundos.
	2. Disponibilidad de los registros: Los registros de auditoría deben estar disponibles al 100% para usuarios autorizados en todo momento.
	3. Integridad de los logs: Ningún registro de auditoría debe ser alterado o eliminado sin dejar un rastro verificable.
	4. Capacidad de almacenamiento: El sistema debe ser capaz de manejar al menos un año de logs sin pérdida de rendimiento, asegurando una recuperación eficiente de los registros.
Prioridad Arquitectónica	Alta
Aplicación	Local

Escenario de Calidad N°2	
ID	QS2
Nombre	Eficiencia de desempeño: Tiempo de respuesta y utilización de recursos.
Sinopsis	Garantizar que la velocidad de carga del servidor sea rápida y estable durante la interacción del usuario, permitiendo cambios de pestaña y la actualización de información sin retrasos perceptibles.
Entorno	Proceso normal de operación del sistema.
Cambio en el entorno	Un aumento en el número de usuarios que acceden simultáneamente al sistema, lo que puede generar una mayor carga en el servidor y afectar la velocidad de respuesta durante las interacciones.

Comportamiento esperado	El sistema debe procesar las solicitudes de cambio de pestaña y de guardado de información de manera rápida, asegurando tiempos de respuesta óptimos y evitando cualquier retraso perceptible en la experiencia del usuario.
Medida	1. Tiempo de respuesta para cambio de pestaña: El sistema debe garantizar un tiempo de respuesta ≤ 1 segundo.
	2. Tiempo de respuesta para guardar información: El sistema debe completar la acción en un tiempo ≤ 3 segundos.
	3. Uso de CPU y memoria: Mantenerse bajo un umbral del 75% durante picos de carga.
Prioridad Arquitectónica	Alta
Aplicación	Local

Escenario de Calidad N°3	
ID	QS3
Nombre	Compatibilidad: Interfaz funcional en múltiples navegadores y sistemas operativos.
Sinopsis	Asegurar que la interfaz del sistema funcione correctamente en diversos navegadores y sistemas operativos, garantizando una experiencia de usuario consistente y accesible.
Entorno	Proceso normal de operación del sistema.
Cambio en el entorno	Un usuario accede a la aplicación desde un navegador o sistema operativo no compatible, lo que puede afectar la funcionalidad y la experiencia del usuario.
Comportamiento esperado	El sistema debe funcionar correctamente y ofrecer una experiencia de usuario sin errores, independientemente del navegador o sistema operativo utilizado, garantizando que todas las funciones sean

	accesibles y visualmente coherentes.
Medida	1. Pruebas de compatibilidad: La interfaz debe ser validada en los principales navegadores (Chrome, Firefox, Safari, Edge) y en los sistemas operativos especificados.
	2. Tasa de error: Cero errores críticos relacionados con la compatibilidad en las pruebas realizadas en diferentes sistemas operativos y navegadores.
	3. Experiencia de usuario: Al menos un 95% de consistencia visual y funcional en las pruebas de interfaz realizadas en diversos dispositivos y sistemas operativos.
Prioridad Arquitectónica	Media
Aplicación	Local

Escenario de Calidad N°4	
ID	QS4
Nombre	Adecuación Funcional: Estabilidad y rendimiento en condiciones de alta carga.
Sinopsis	Garantizar que el sistema mantenga todas sus funcionalidades operativas y estables durante picos de uso y alta carga, sin degradación en el rendimiento ni pérdida de funcionalidad.
Entorno	Proceso normal de operación del sistema.
Cambio en el entorno	Un incremento repentino en el número de usuarios que utilizan la aplicación al mismo tiempo, puede llevar a una mayor demanda de recursos y potencialmente afectar la capacidad de respuesta del sistema.

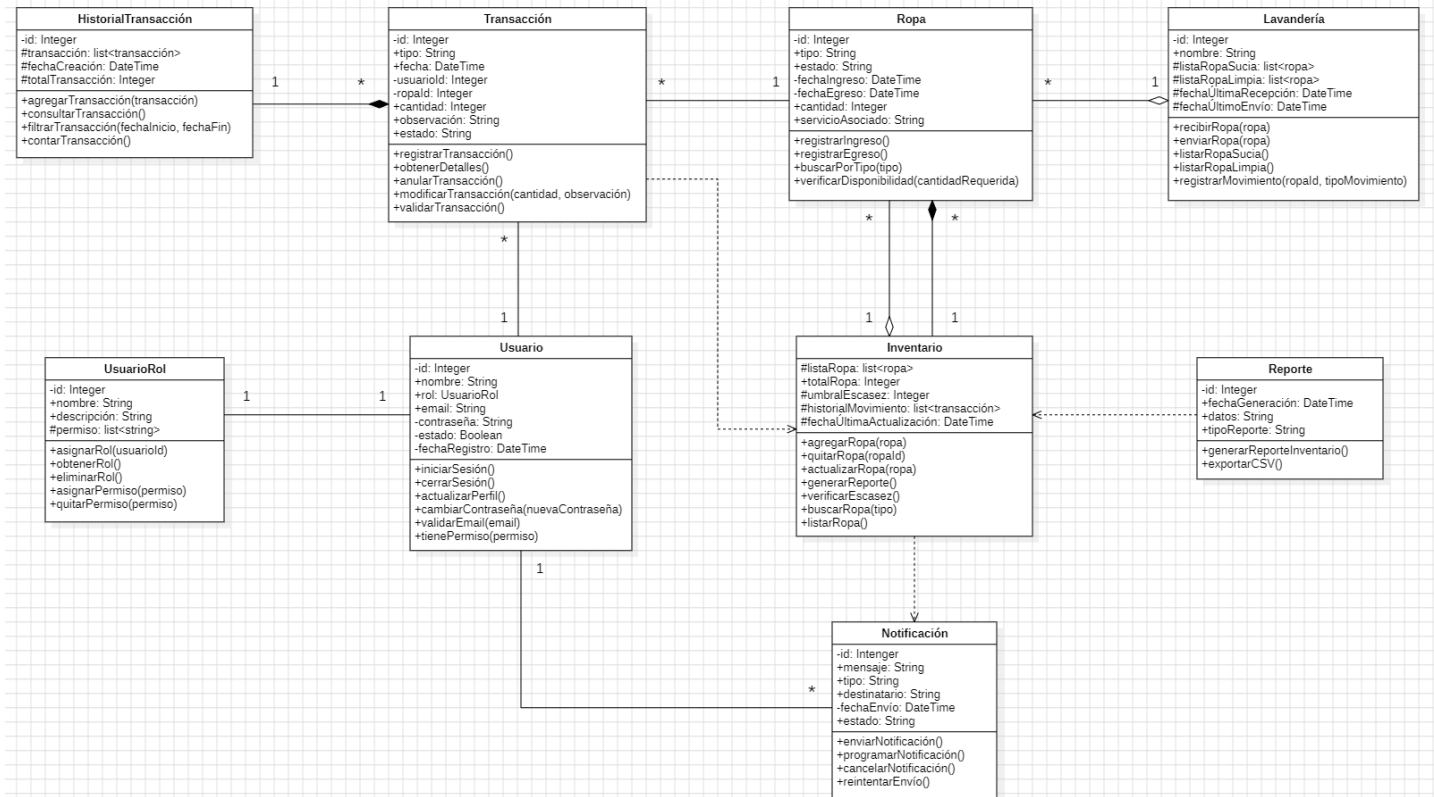
Comportamiento esperado	El sistema debe operar de manera continua y estable, preservando la integridad de sus servicios sin interrupciones, incluso durante períodos de alta demanda.
Medida	1. Tasa de disponibilidad: $\geq 99.5\%$ durante periodos de alta carga.
	2. Tiempo de respuesta: ≤ 3 segundos en condiciones normales y ≤ 5 segundos durante picos de uso.
	3. Mantenimiento de funcionalidad: El 100% de las funciones críticas deben seguir operativas bajo todas las condiciones de carga.
	4. Tasa de error: La tasa de transacciones fallidas o interrumpidas no debe exceder el 0.1% durante momentos de alta demanda.
Prioridad Arquitectónica	Media
Aplicación	Local

Escenario de Calidad N°5	
ID	QS5
Nombre	Seguridad: Protección de datos y acceso restringido
Sinopsis	Asegurar que los datos del inventario sean accesibles sólo para usuarios autorizados mediante encriptación y políticas de autenticación, garantizando la confidencialidad.
Entorno	Simulación de un entorno de prueba que imita situaciones reales de acceso no autorizado para evaluar la seguridad de los datos.
Cambio en el entorno	Un incremento en los intentos de acceso no autorizados a los datos del inventario, podría poner a prueba la eficacia de las medidas de seguridad implementadas.
Comportamiento	Los datos del inventario deben estar encriptados y accesibles únicamente para usuarios autorizados. Cualquier intento de acceso no

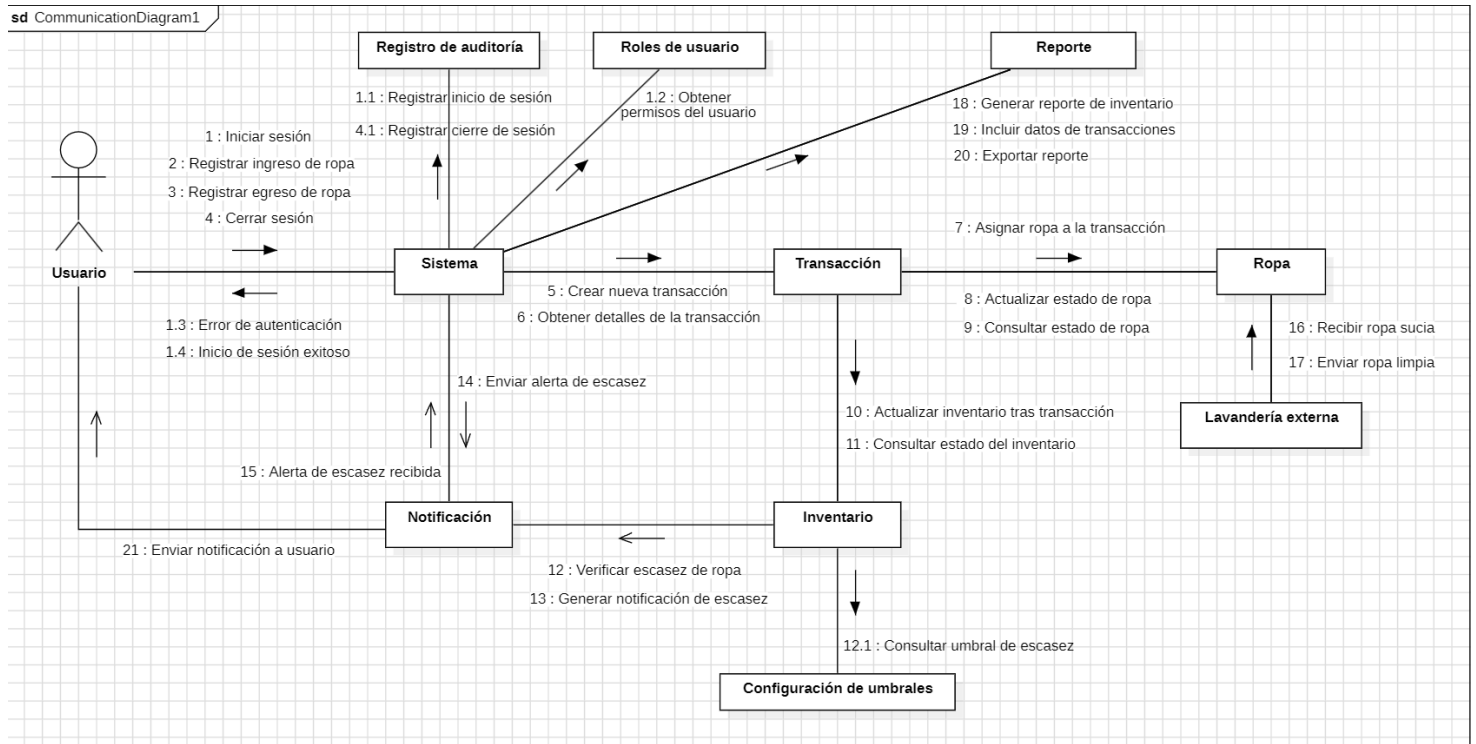
esperado	autorizado debe ser bloqueado y registrado, garantizando la confidencialidad y la integridad de la información.
Medida	1. Tiempo de acceso a datos encriptados: Debe ser ≤ 3 segundos para usuarios autorizados.
	2. Acceso no autorizado: 0% de accesos no autorizados o violaciones de seguridad registradas.
	3. Registro de intentos de acceso: Todos los intentos de acceso no autorizado deben ser registrados y alertar a los administradores.
Prioridad Arquitectónica	Alta
Aplicación	Local

Vista Lógica

- **Diagrama de Clases:**

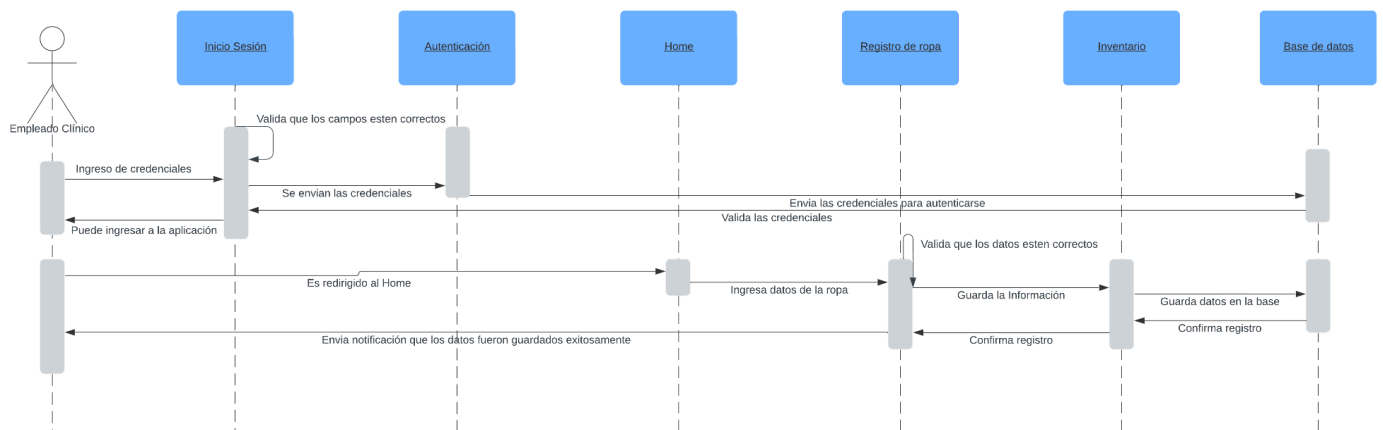


- **Diagrama de Comunicación:**



- **Diagrama de Secuencia:**

R1: Registro de ingreso de ropa



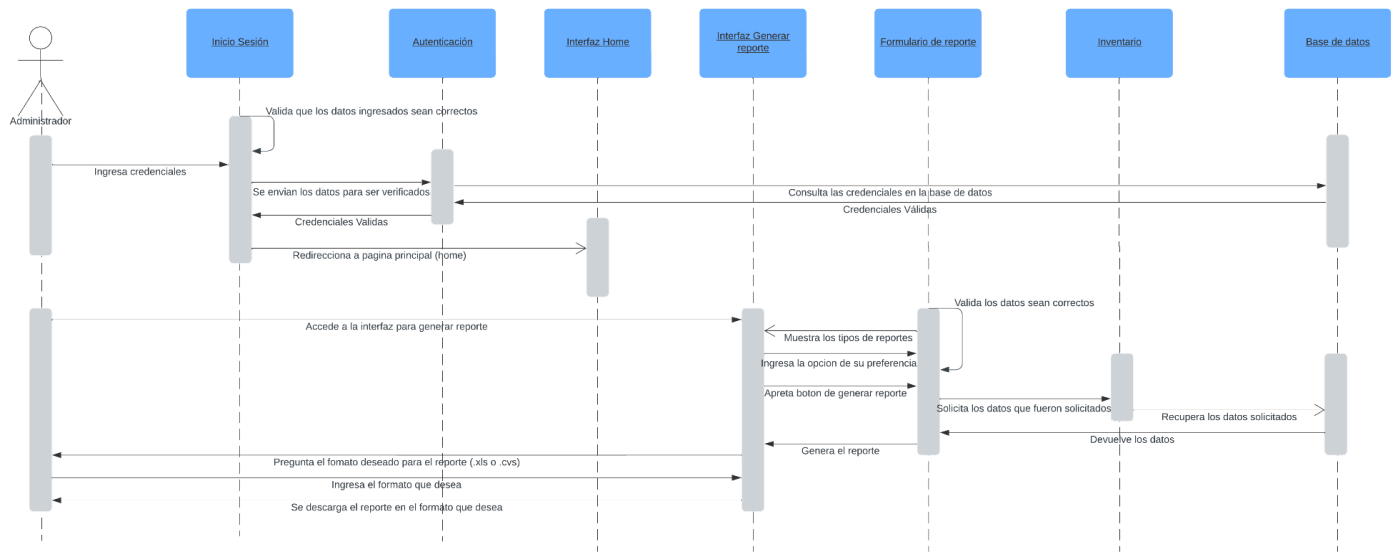
Documento Confidencial



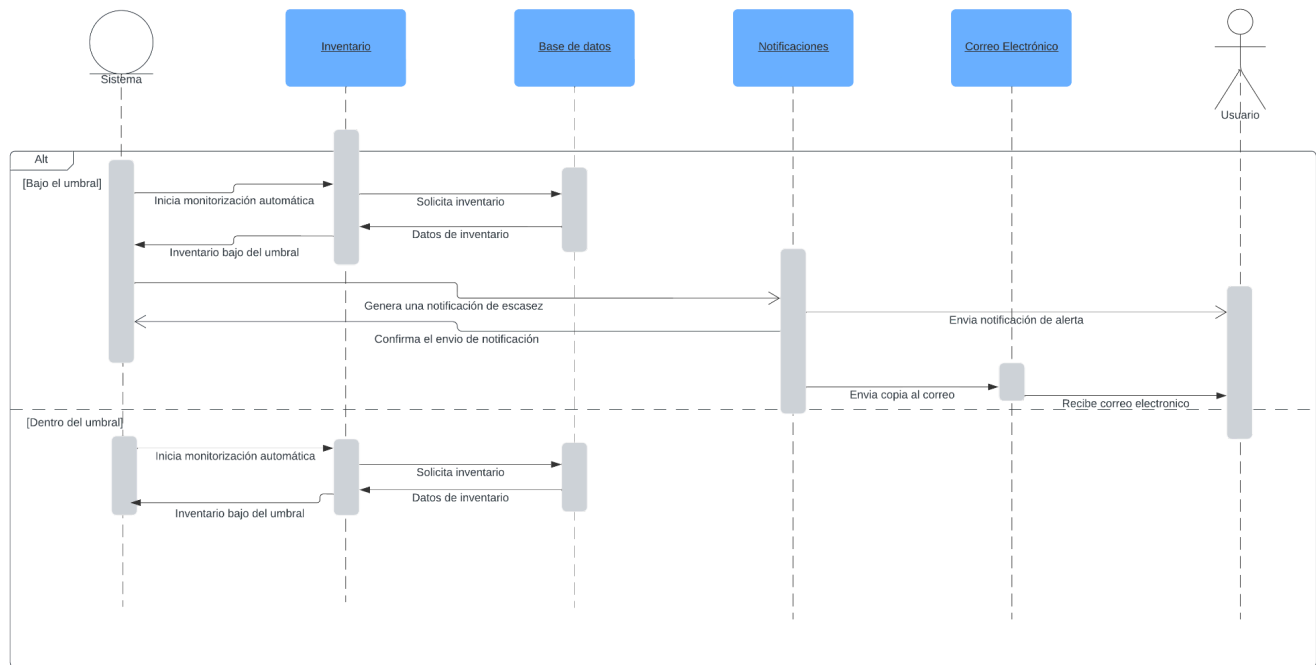
Documento Confidencial



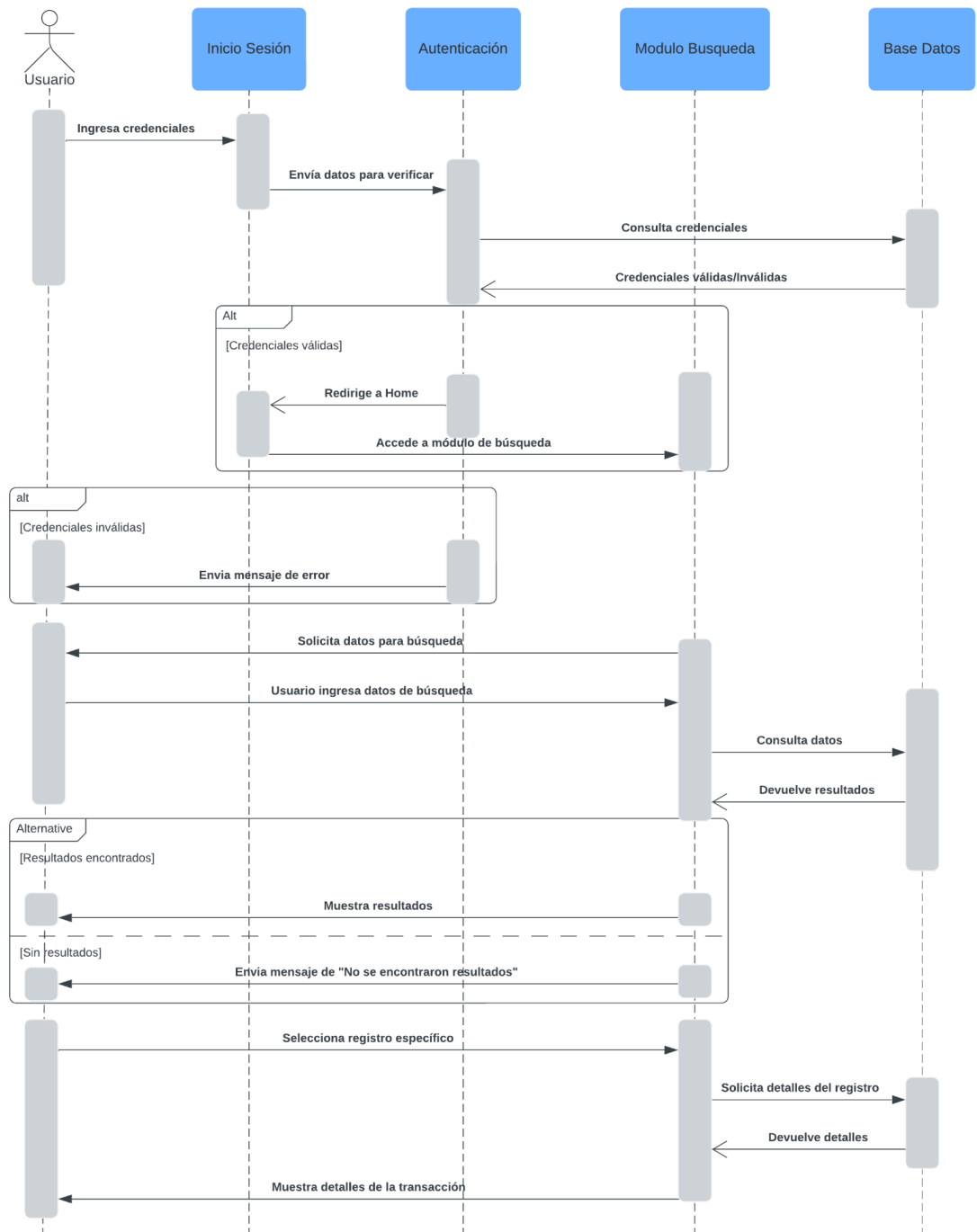
R4: Generar reportes de inventario



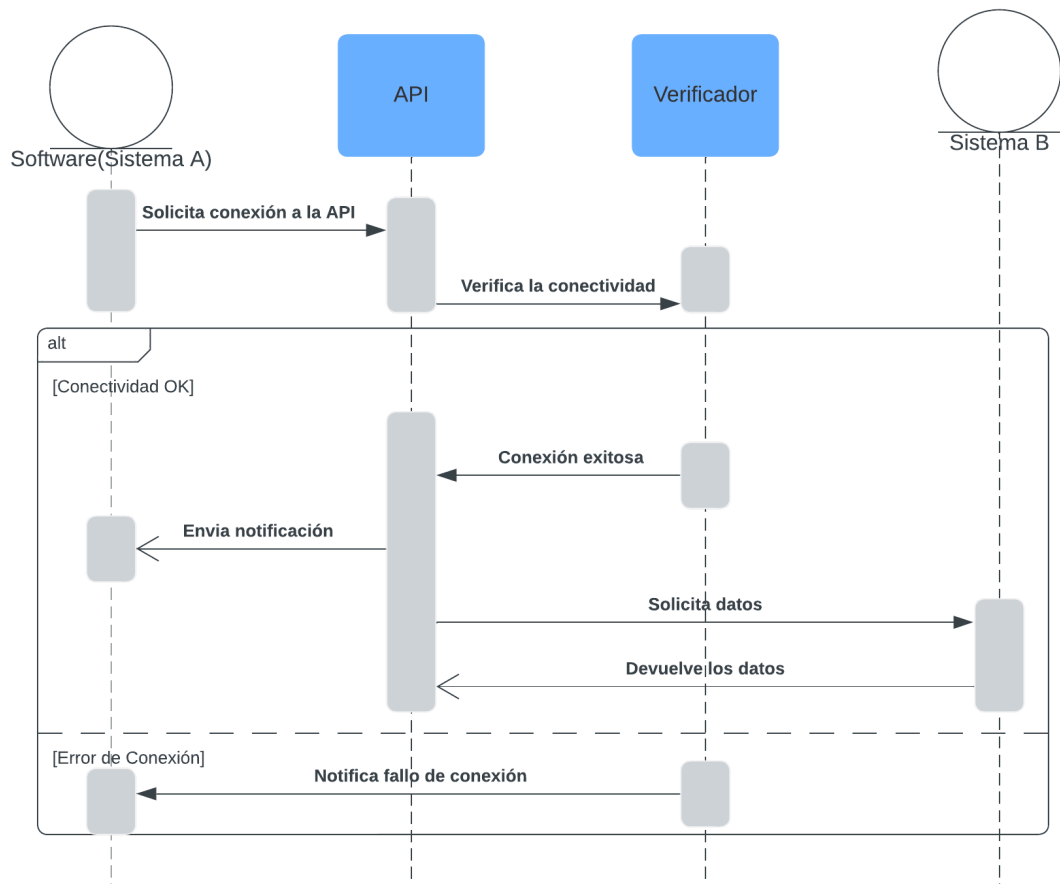
R5: Notificación de escasez de ropa



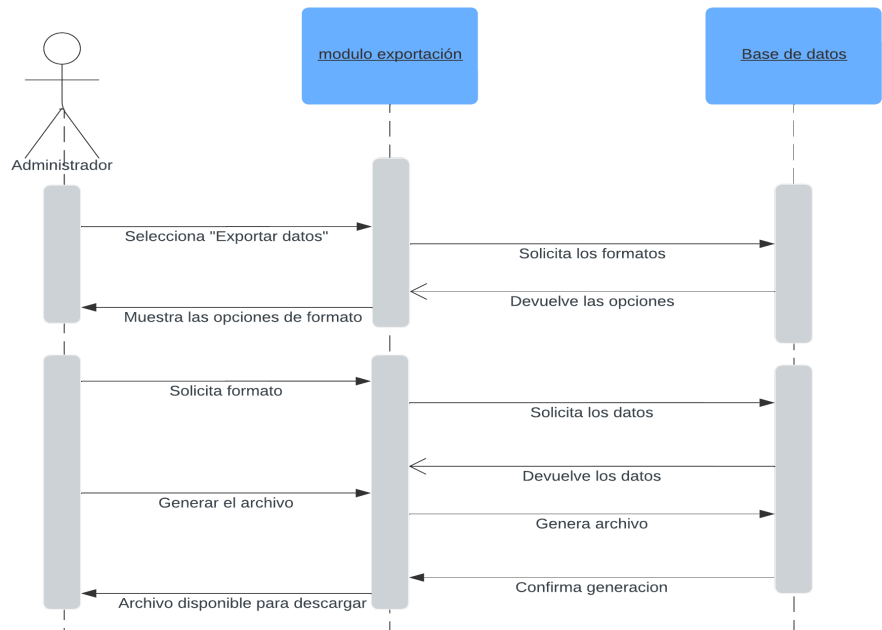
R6: Herramienta de búsqueda y seguimiento de transacciones



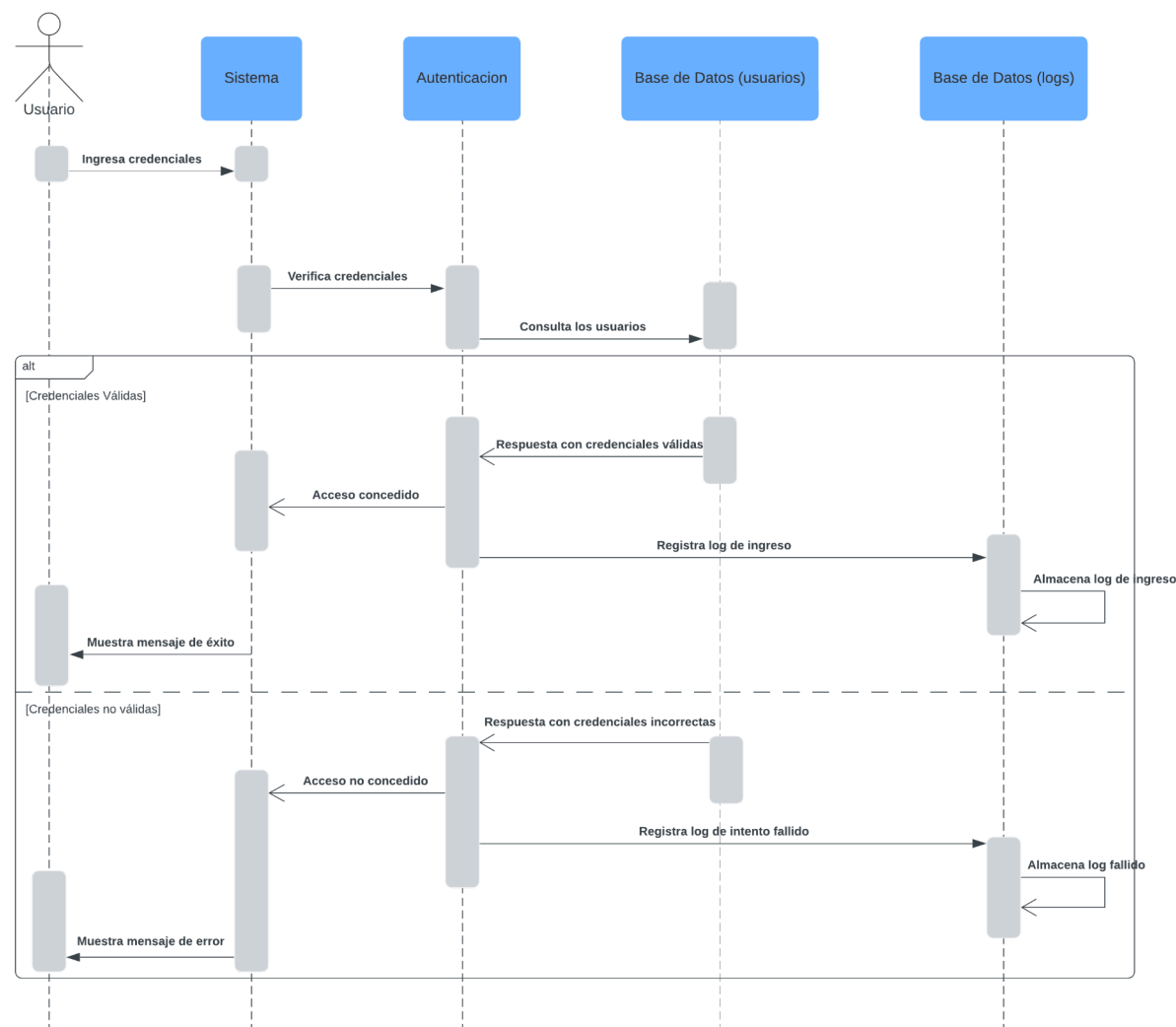
R7: Integración con sistemas existentes



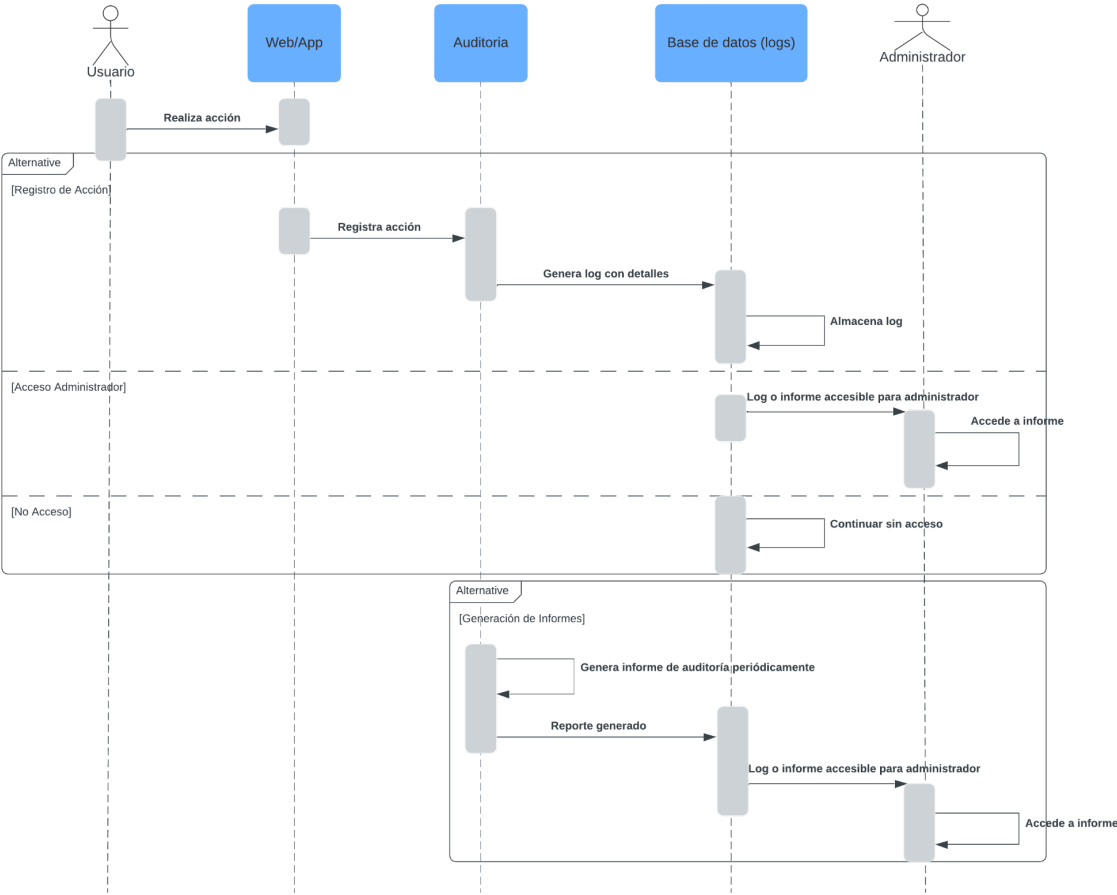
R8: Soporte para exportación de datos



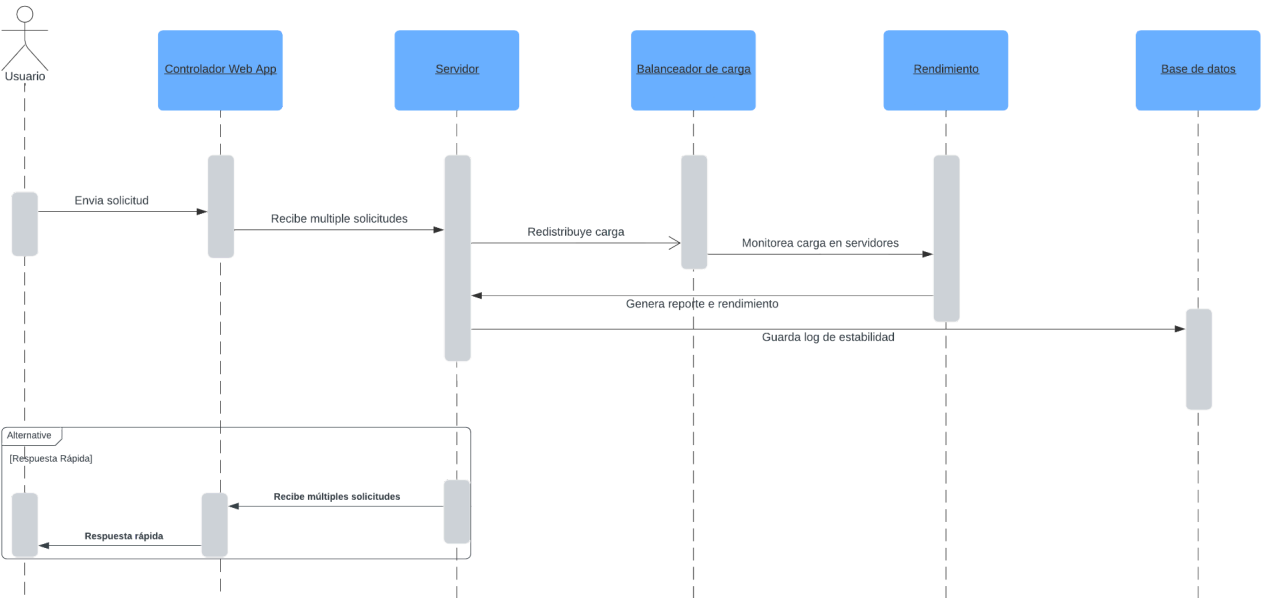
R9: Seguridad de accesibilidad de datos



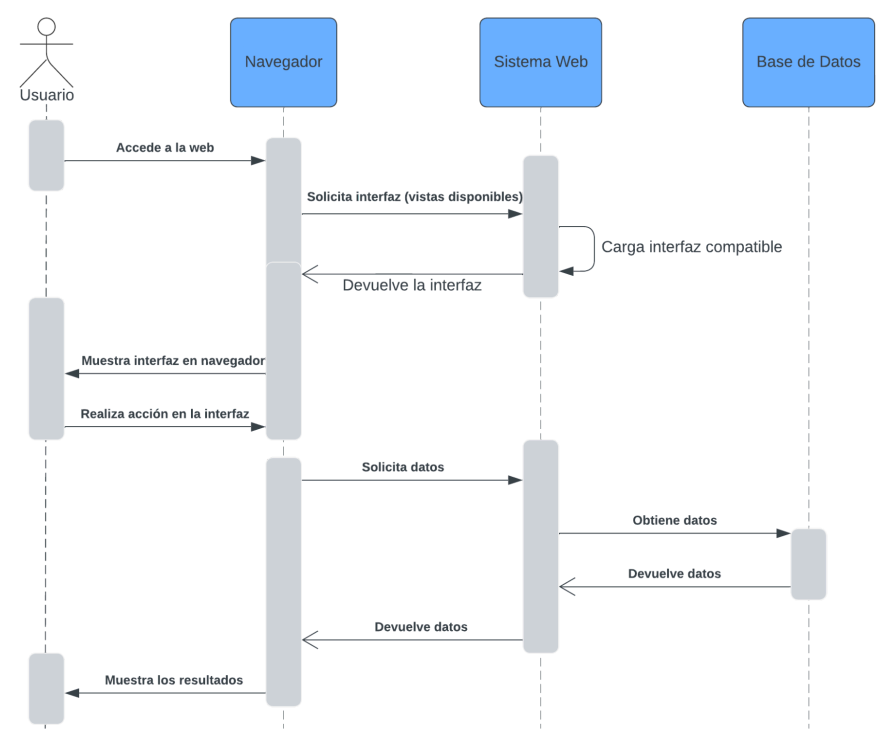
R10: Registro de auditoría y logs de actividad



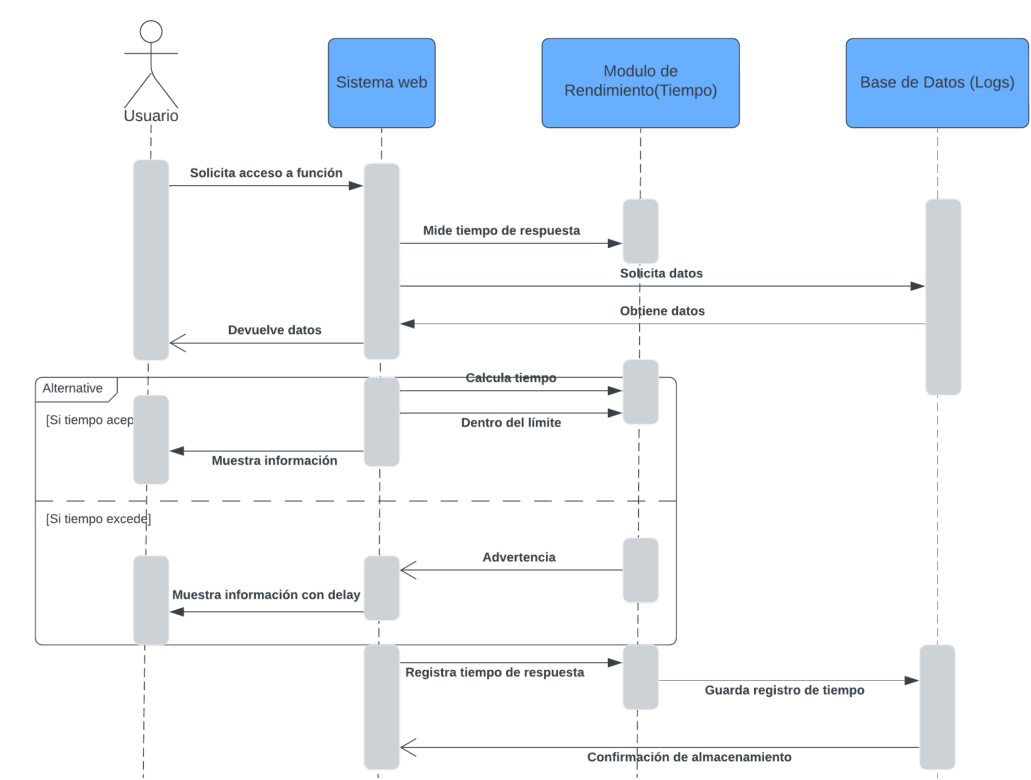
R11: Estabilidad de funcionalidad de la aplicación



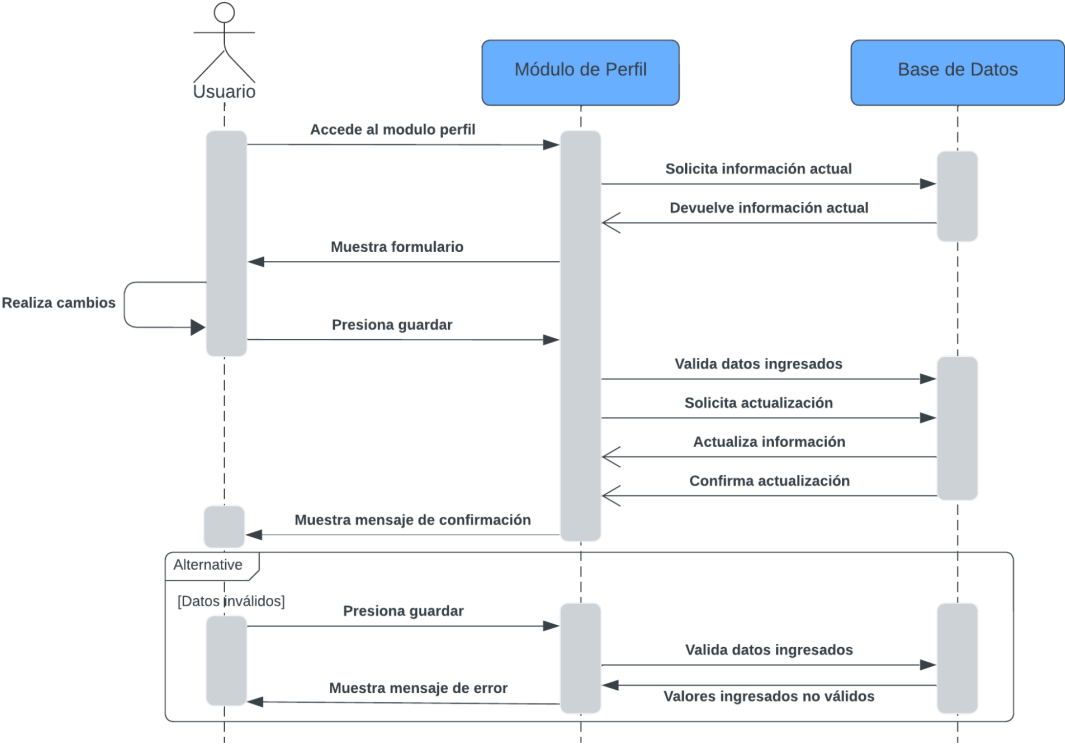
R12: Interfaz compatible con navegadores



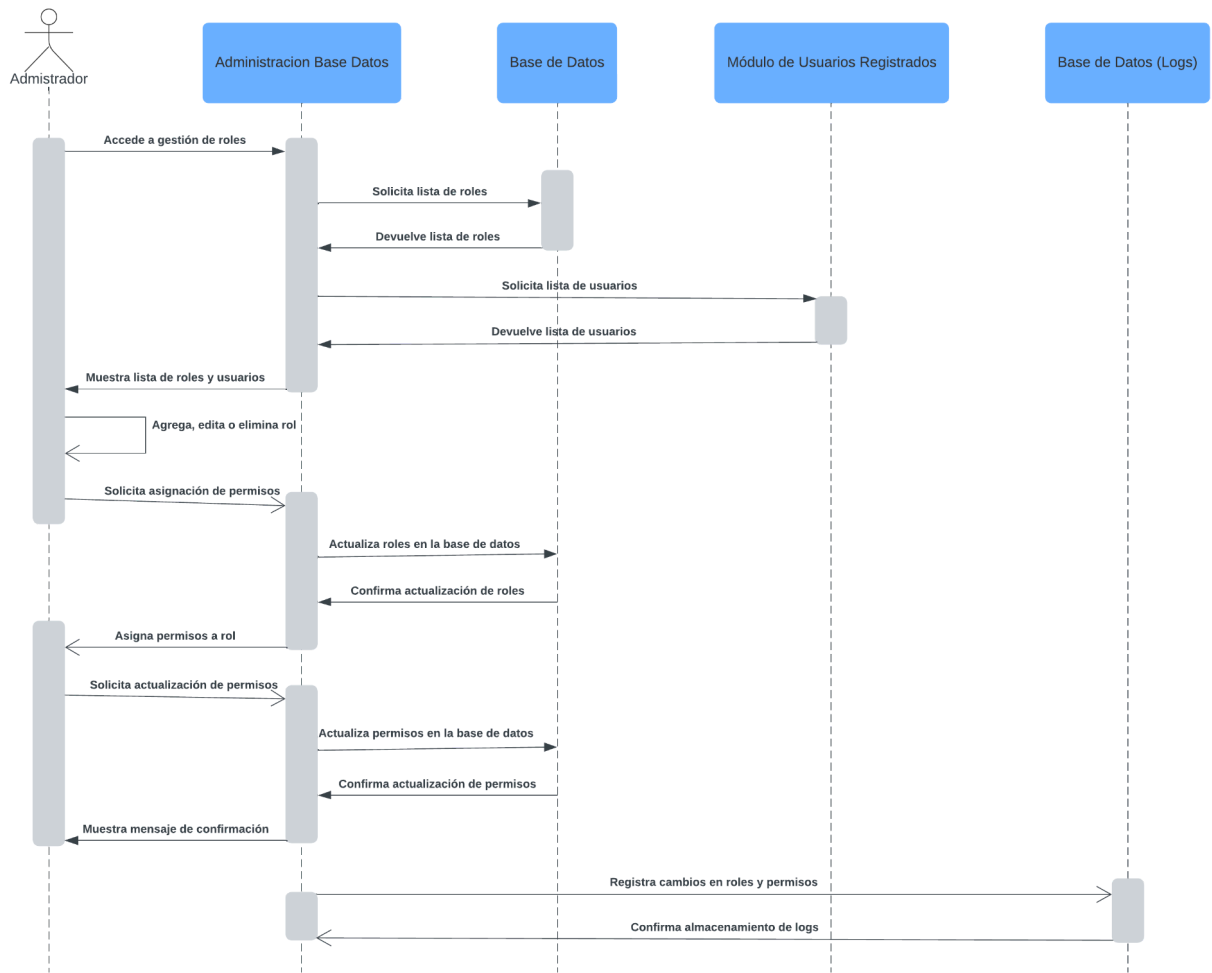
R13: Velocidad de carga rápida y estable



R14: Actualización de Perfil de Usuario

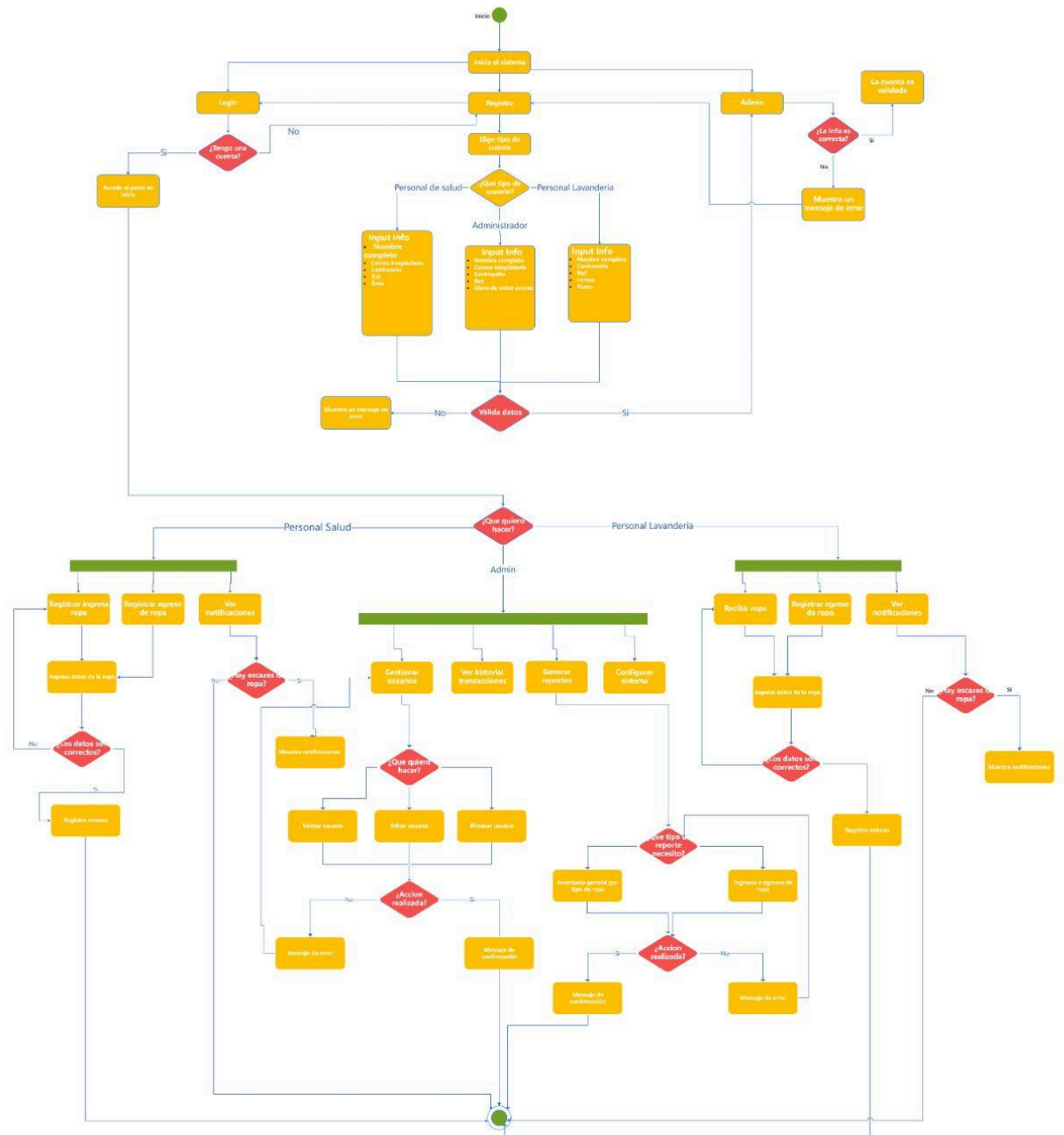


R15: Gestión de Roles y Permisos



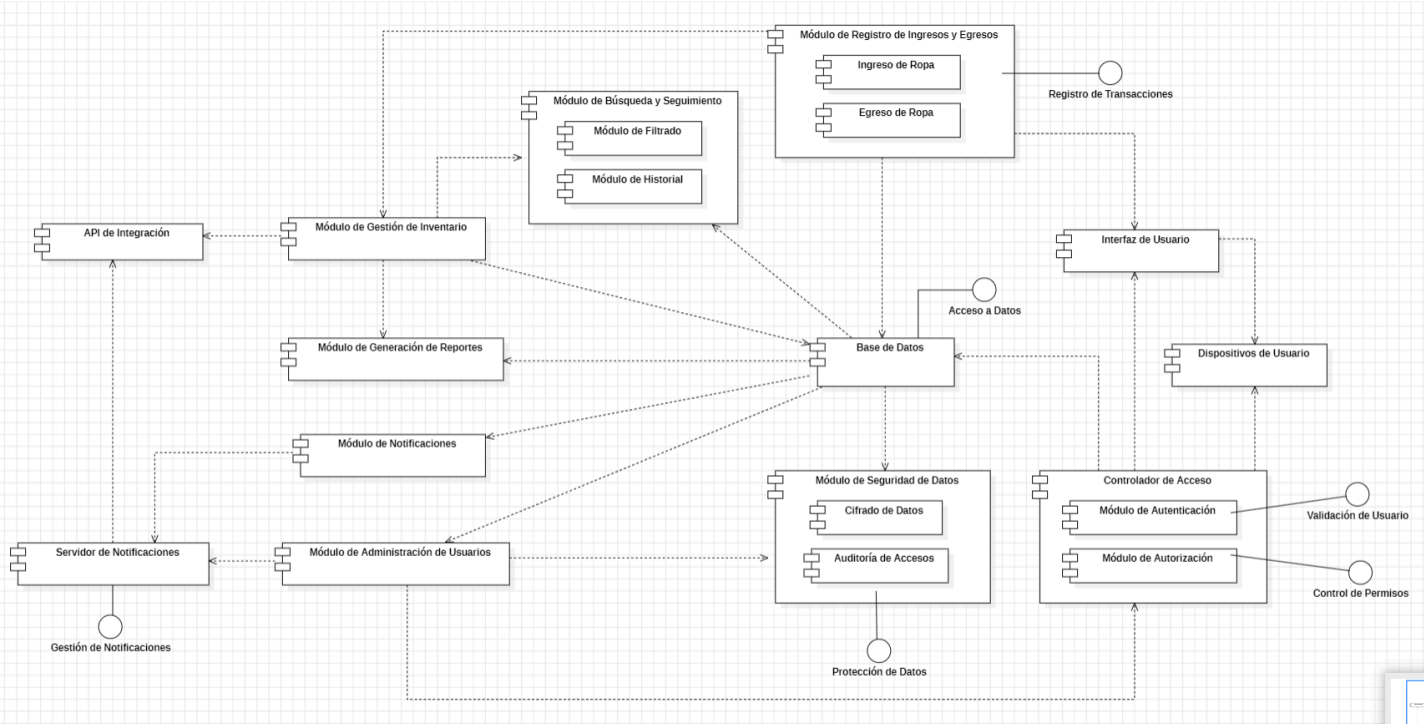
Vista de Procesos

- **Diagrama de Actividad:**

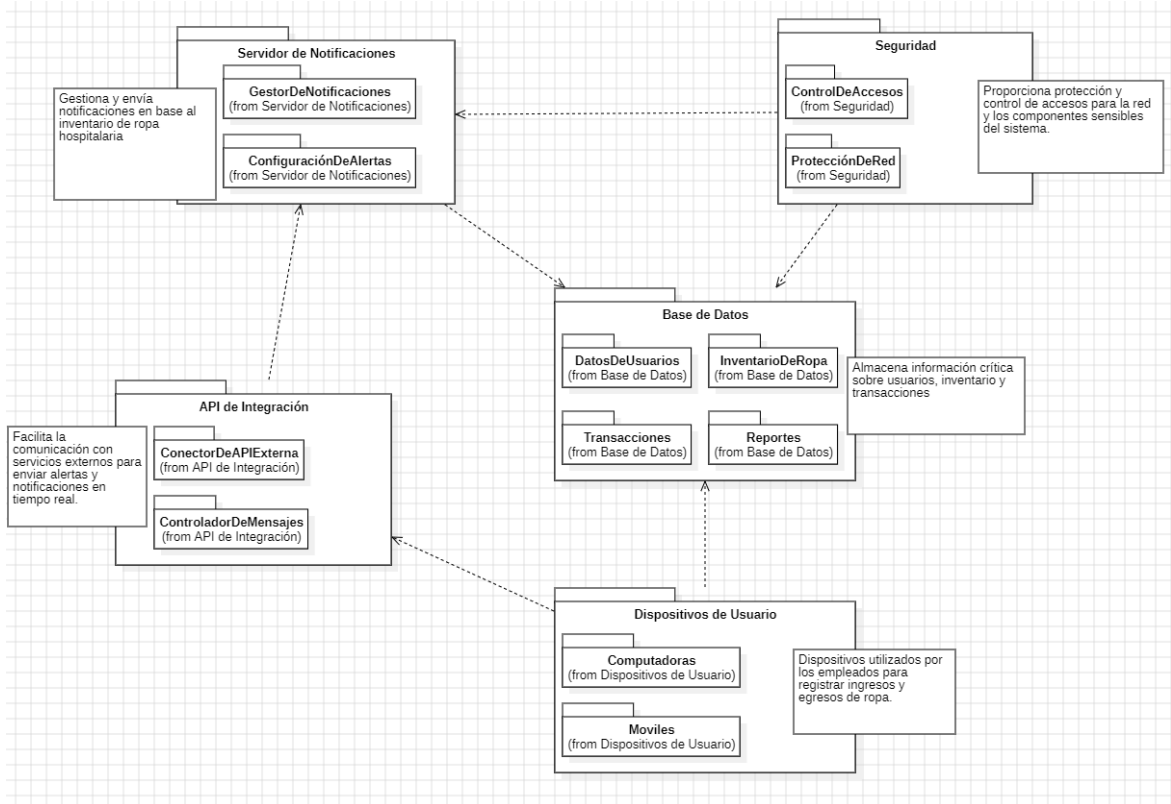


Vista de Despliegue

- Diagrama de Componentes:

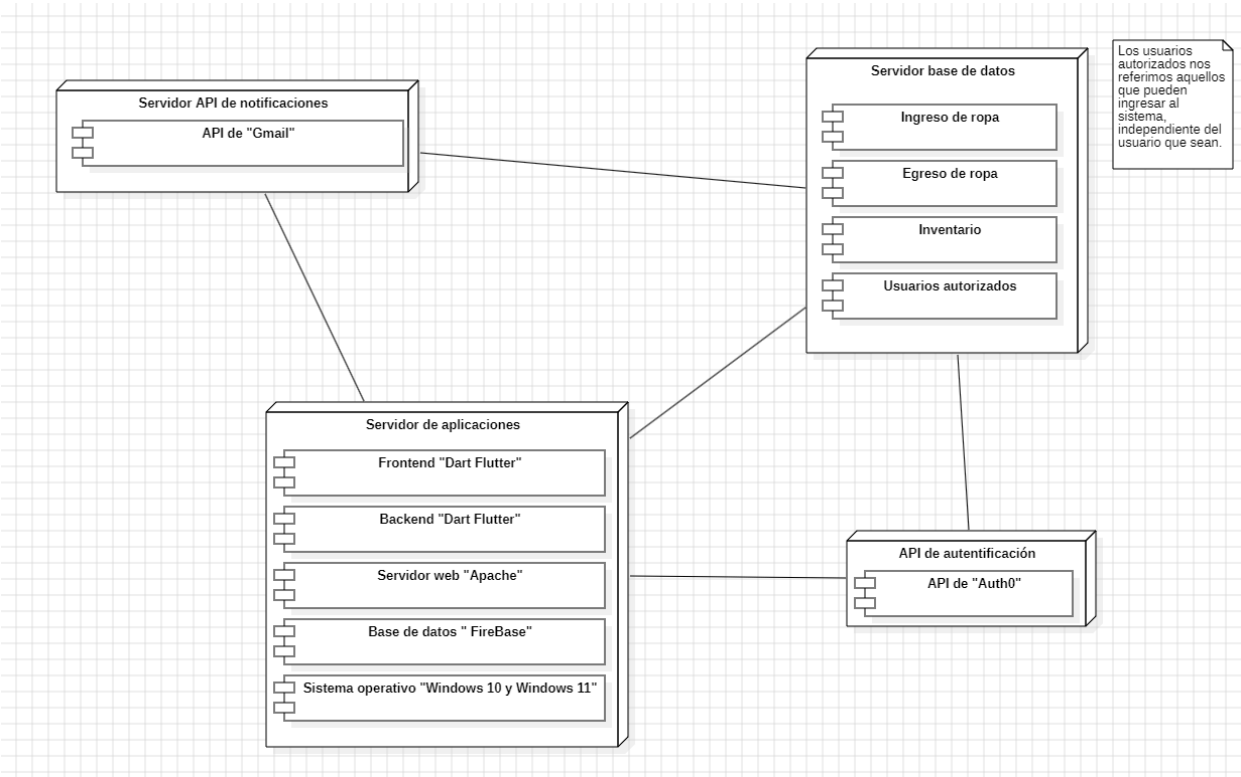


- **Diagrama de Paquetes:**

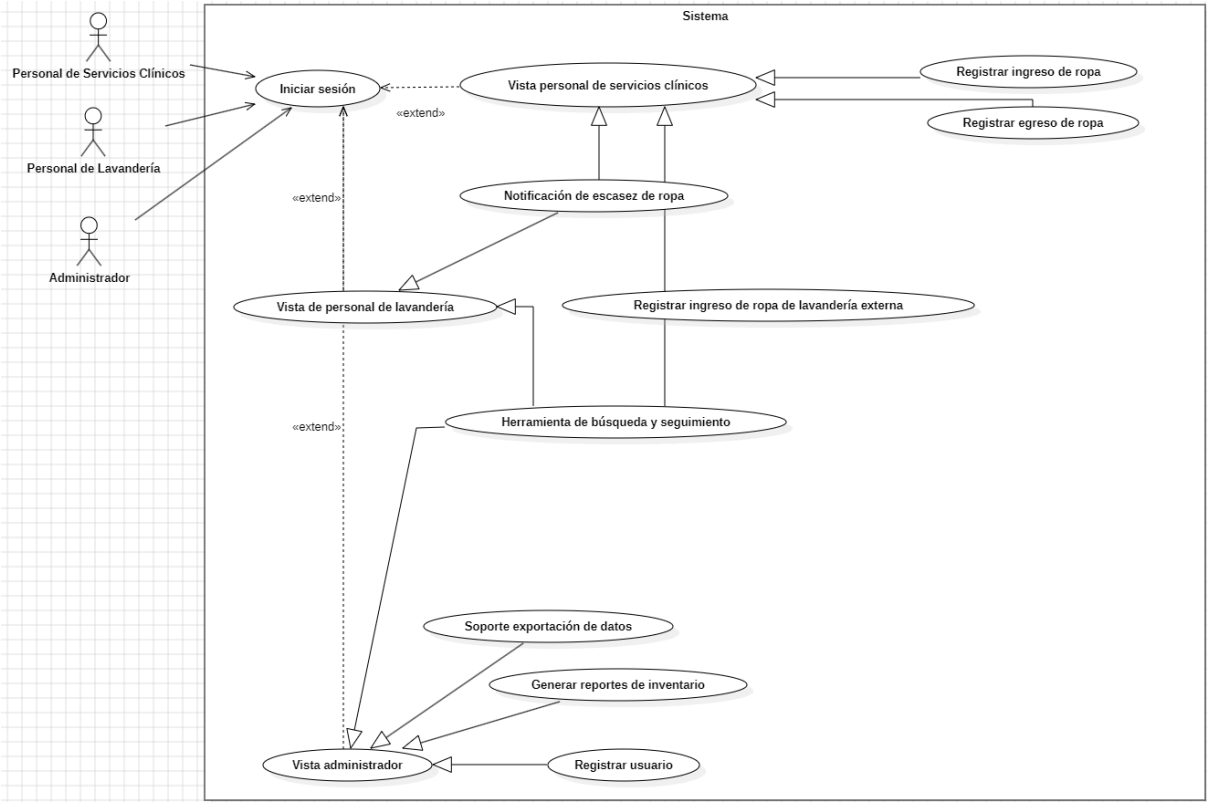


Vista Física

- **Diagrama de Despliegue:**



Vista Escenarios 4 + 1



- **Decisiones de Diseño y Selección de Alternativas**

Las decisiones arquitectónicas clave para la aplicación web de gestión de ropa hospitalaria se tomaron teniendo en cuenta la restricción de **tiempo de construcción**. Por ello, se optó por un diseño modular que divide la arquitectura en componentes independientes, cada uno encargado de funciones específicas.

Además, se implementará el “**Patrón Provider**” para la gestión de datos, lo que permitirá al sistema delegar de manera eficiente las solicitudes de información a servicios externos. Este patrón es fundamental para la arquitectura de la aplicación, ya que proporciona un mecanismo centralizado para el manejo del estado, permitiendo que diversos componentes de la interfaz de usuario accedan a los datos de manera coherente y reactiva.

Su uso facilitará la integración de funcionalidades clave del caso, como las notificaciones de escasez de ropa y el seguimiento de transacciones. Por ejemplo, el sistema podrá suscribirse a cambios en el estado del inventario, enviando alertas automáticas a los usuarios cuando los niveles de ropa caigan por debajo de un umbral predefinido. Esto asegura una respuesta ágil ante situaciones críticas, mejorando así la eficiencia operativa del hospital.

En este contexto, se seleccionaron **Flutter y Dart** como tecnologías de desarrollo, lo que se alinea con las decisiones arquitectónicas ya mencionadas. La capacidad de Flutter para crear aplicaciones multiplataforma a partir de una única base de código es fundamental para optimizar el tiempo de desarrollo del proyecto. Esta característica permite al equipo escribir el código una sola vez y desplegarlo en diversas plataformas, lo que no sólo acelera el proceso, sino que también asegura una experiencia de usuario coherente en todos los dispositivos.

Además, la funcionalidad de “**hot reload**” permite visualizar instantáneamente los cambios en el código sin necesidad de reiniciar la aplicación, lo que facilita una rápida adaptación a los requerimientos del usuario, permitiendo implementar modificaciones y mejoras de forma ágil y efectiva. Por último, el amplio ecosistema de paquetes disponibles en **pub.dev** facilitará la integración de componentes críticos, como la gestión del estado y la autenticación de usuarios, asegurando que la aplicación cumpla con los requisitos de seguridad y accesibilidad establecidos en los escenarios de calidad.

5. Análisis de Reutilización

El objetivo de la reutilización es identificar y estructurar componentes y funcionalidades susceptibles de reutilización en toda la aplicación. Esta estrategia optimiza el ciclo de desarrollo al reducir la duplicación de esfuerzos, mejora la calidad y lectura del código y facilita el mantenimiento a largo plazo del sistema.

A continuación, se detallan los aspectos relevantes:

1. Identificación de componentes reutilizables

- Componente de notificación: Se desarrollará un “**widget modular**” diseñado para alertar a los usuarios sobre situaciones críticas, como la escasez de ropa en los servicios clínicos. Este componente es esencial para el funcionamiento efectivo del sistema de gestión de inventario y permite mantener informado al personal médico y a los responsables de la lavandería en tiempo real.

Características

- Adaptabilidad: Personalizable para enviar diferentes tipos de notificaciones (escasez de ropa limpia, reabastecimiento necesario, atención urgente para ropa sucia).
- Integración con APIs: Capaz de conectarse a servicios de mensajería como **SendGrid** o **Twilio**, permitiendo el envío de notificaciones a través de **correo electrónico** o **dentro de la misma aplicación**.

2. Funcionalidades comunes

- Gestión de datos: Se implementará una función unificada para el registro de ingresos y egresos de ropa hospitalaria, lo que mitiga la “**duplicación de código**” y respalda una lógica coherente en todo el sistema.

Características

- Unificación de código: Una única función para registrar ambos tipos de transacciones simplificará el mantenimiento y la actualización de la lógica del sistema.
- Coherencia en la lógica: Aplicación de las mismas reglas y validaciones para ambos registros, asegurando la integridad de los datos.
- Facilidad de escalabilidad: Permite incorporar nuevas características o tipos de ropa en el futuro de manera eficiente.

3. Uso de librerías externas

- Integración de paquetes: Se implementará el uso de librerías de la comunicación de Flutter, como el método **http**, para gestionar solicitudes de red a la API del sistema de gestión de inventario.

Características

- Manejo de respuestas: Simplificación de la gestión de datos mediante la deserialización de **JSON** en objetos Dart.
- Gestión de errores: Inclusión de herramientas para manejar errores y excepciones, asegurando una respuesta adecuada ante problemas.